

Python Study Notes (Part 2)

These notes continue the workspace topics with the same flow: explanation → code → explanation.

functions.py — Functions, args, kwargs

Functions can return values and accept variable arguments.

```
def sum_numbers(a, b):  
    if not isinstance(a, int) or not isinstance(b, int):  
        return None  
    return a + b  
  
def multiple_items(*args):  
    print(args)  
  
def keyword_arguments(**kwargs):  
    print(kwargs)
```

Output:

```
(no output; functions are defined)
```

`*args` collects positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict. Returning `None` is a common way to signal invalid input.

lambda.py — Lambdas, map/filter/reduce

Lambdas are small, anonymous functions.

```
numbers = [1, 2, 3, 4, 5]  
squares = list(map(lambda n: n * n, numbers))  
odds = list(filter(lambda n: n % 2 != 0, numbers))
```

Resulting values:

```
squares = [1, 4, 9, 16, 25]  
odds = [1, 3, 5]
```

`map` transforms each item; `filter` keeps items that pass a condition. Prefer `def` when logic becomes complex.

scope.py — LEGB, nonlocal

Python resolves names using LEGB: Local → Enclosing → Global → Built-in.

```
def outer():  
    color = "green"  
    def inner():  
        nonlocal color  
        color = "blue"  
    inner()  
    return color  
  
print(outer())
```

Output:

```
blue
```

`nonlocal` lets you rebind a variable from the enclosing scope, so `outer` returns `"blue"`.

closure.py — Closures

A closure is a function that remembers variables from its outer scope.

```
def counter_factory(start=0):  
    count = start  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
counter = counter_factory(10)  
print(counter(), counter())
```

Output:

```
11 12
```

Each call to `counter_factory` creates an independent counter with its own state.

recursive.py — Recursion

Recursion needs a base case to stop.

```
def factorial(n):  
    if n < 0:  
        raise ValueError("n must be >= 0")  
    if n in (0, 1):  
        return 1  
    return n * factorial(n - 1)
```

Output:

(no output; function is defined)

`factorial(5)` returns `120`. Without a base case, recursion never ends.

classes.py — Classes and inheritance

Subclasses can override parent methods and still reuse base behavior.

```
class Vehicle:  
    def __init__(self, make, model):  
        self.make = make  
        self.model = model  
    def moves(self):  
        print("The vehicle moves")  
  
class Boat(Vehicle):  
    def moves(self):  
        print("The boat sails")
```

Output:

(no output; classes are defined)

`Boat` inherits from `Vehicle` but overrides `moves` to change behavior.

bank.py + oop_proj.py — Exceptions and OOP usage

Custom exceptions separate error flow from success flow.

```
class BalanceError(Exception):  
    pass  
  
class BankAccount:  
    def __init__(self, initial_amount, acct_name):  
        self.balance = float(initial_amount)
```

```
        self.acct_name = acct_name
    def withdraw(self, amount):
        if self.balance < amount:
            raise BalanceError("Insufficient funds")
        self.balance -= amount
```

Output:

(no output; classes are defined)

In `oop_proj.py`, you import these classes and call methods to simulate account activity.

modules.py + newyork.py — Imports and **name**

Modules can expose constants and functions and still be runnable scripts.

```
# newyork.py
capital = "Albany"
def random_city():
    return "Albany"

# modules.py
import newyork
print(newyork.capital)
```

Output:

Albany

When a file is run directly, `__name__ == "__main__"` is True; when imported, `__name__` is the module name.

extensions.py — pathlib for file paths

`pathlib` builds paths safely across OSes.

```
from pathlib import Path
base = Path("projects") / "demo"
file_path = base / "data.csv"
print(file_path.with_suffix(".json"))
```

Output:

```
projects/demo/data.json
```

`with_suffix` swaps extensions without manual string manipulation.

hello_person.py — CLI with argparse

Command-line interfaces can validate input and show help text.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--name", required=True)
args = parser.parse_args()
print(f"Hello, {args.name}!")
```

Output (example run):

```
Hello, Dave!
```

Running `python3 hello_person.py --name Dave` prints a personalized greeting.

rps.py / rps2.py / rps3.py — Rock Paper Scissors evolution

Start with a single round, then add loops and match scoring.

```
from enum import Enum
class RPS(Enum):
    ROCK = 1
    PAPER = 2
    SCISSORS = 3

def decide_winner(player, computer):
    wins = {RPS.ROCK: RPS.SCISSORS, RPS.PAPER: RPS.ROCK, RPS.SCISSORS:
RPS.PAPER}
    if player == computer:
        return "tie"
    return "win" if wins[player] == computer else "lose"
```

Output:

```
(no output; function is defined)
```

`rps2.py` adds a play-again loop and a score. `rps3.py` adds a best-of match.